

Код-ревью

RTSP Gateway — шлюз RTSP → HTTPS

Ветка / коммит	main · 2a5678c «fixed more bags, 0.5v»	Дата	7 сентября 2026
Объём проверки	49 модулей Python (9 486 строк), 20 модулей тестов (3 916 строк), 25 шаблонов, 2 287 строк JS/CSS, 2 205 строк shell, инфраструктура (Caddyfile, compose, mediamtx.yml, Dockerfile, CI)	Метод	Сплошное чтение исходников + запуск линтеров, типов, тестов и аудита зависимостей

1. Краткий обзор

Код существенно выше среднего уровня для проектов этого класса. Архитектура продумана и, что важнее, **объяснена**: почти каждое неочевидное решение сопровождается комментарием, который отвечает не «что делает эта строка», а «почему именно так и что ломалось раньше». Это редкость, и это главный актив проекта — такой код можно передать другому человеку без потери контекста.

Безопасность построена на правильных основаниях, а не на затычках: серверные сессии в Redis вместо подписанных cookie, проверка прав на **каждый** медиа-запрос через forward_auth, шифрование учётных данных камер SecretBox, argon2id с усиленными параметрами, защита TOTP от повторного использования кода, ротация идентификатора сессии при смене уровня привилегий, единая точка SSRF-проверки для RTSP и PTZ. Сетевая схема docker разделена на четыре сети с внятной мотивацией каждой.

Все автоматические проверки проекта проходят. Найденные дефекты — не архитектурные: это точечные упущения в редких ветках кода и несколько мест, где нагрузка вырастет быстрее, чем ожидается. **Ни одной уязвимости, дающей обход аутентификации, доступ к чужой камере или утечку секретов, не обнаружено.**

Результаты автоматических проверок

Проверка	Команда	Результат
Линтер	ruff check app alembic tests	чисто — 0 замечаний при наборе правил E, F, I, UP, B, S, ASYNC, RUF
Типы	mypy app	чисто — 49 файлов, режим strict
Тесты	pytest -q	450 прошли, 10 упали — падения не в коде: в этой рабочей копии fakeredis установлен без extra [lua], и тесты блокировки PTZ не могут выполнить EVAL. Лечится pip install -e ".[dev]"; в CI extra ставится и тесты проходят
Зависимости	pip-audit --desc	1 находка — PYSEC-2026-1845 в pytest 8.4.2. Это dev-зависимость, в образ она не попадает. Подробности — S2. Рантайм-зависимости чисты.

Оценка по запрошенным аспектам

Аспект	Оценка	Комментарий
Качество кода и практики Python	отлично	Соответствует лучшим практикам. Типизация строгая и полная, современный синтаксис 3.12 (type-алиасы, PEP 695-дженерики, StrEnum, slots=True), чистое разделение слоёв, отсутствие дублирования. Замечаний уровня «так писать не следует» нет
Баги и граничные случаи	хорошо	6 находок, все локальные. Одна (B1) реально отключает заявленное ограничение на число зрителей, остальные — низкой важности
Производительность	удовлетворительно	5 находок. Две (P1, P2) проявятся не в лаборатории, а на боевой нагрузке: полный скан Redis каждые 15 с и блокировка фонового цикла внешними процессами
Безопасность	хорошо	Непреднамеренных уязвимостей с существенным влиянием нет. 8 находок: цепочка поставок (S1), просроченный пин зависимости (S2), латентные и эшелонированные (S4, S5, S7). Осознанные компромиссы вынесены в раздел 6

2. Сводка находок

19 находок. Критических и высоких нет. Приоритет к исправлению — по колонке «Важность»; раздел 6 отдельно перечисляет компромиссы, которые сделаны намеренно и задокументированы в самом коде — они **не** являются дефектами.

ID	Находка	Важность	Что это значит на практике
B1	TTL множества зрителей ссылки не продлевается	СРЕД	Через час непрерывного просмотра лимит одновременных зрителей перестаёт действовать, а метрика занижает число зрителей
B2	Блокировка PTZ не снимается при неизвестном направлении	НИЗК	Камера остаётся занятой на 15 с после запроса с мусорным направлением
B3	Счётчик просмотров ссылки инкрементируется гонкой	НИЗК	Одновременные открытия теряют часть инкрементов
B4	Ответ камеры читается целиком, ограничение мнимое	НИЗК	Камера с огромным ответом раздувает память процесса <code>api</code>
B5	Гонка при одновременном принятии приглашения	НИЗК	Вместо понятного сообщения сотрудник видит страницу 500
B6	Заблокированная учётка отличима по времени ответа	ИНФО	Подтверждает существование учётной записи; плюс блокировку можно удерживать извне
P1	<code>/metrics</code> сканирует весь <code>keyspace</code> Redis	СРЕД	Каждые 15 с полный обход всех ключей Redis — растёт вместе с числом сессий
P2	Цикл <code>worker'a</code> блокируется <code>ffprobe</code> и <code>ffmpeg</code>	СРЕД	Реконсильция и статусы камер простаивают до двух минут вместо 15 с
P3	Списки камер, пользователей и ссылок без пагинации	НИЗК	Страницы деградируют по мере роста инсталляции
P4	Разбор CIDR на каждом медиа-запросе	НИЗК	Лишняя работа несколько раз в секунду на каждого зрителя
P5	Новый HTTP-клиент на каждый зонд диагностики	НИЗК	Мелкая неэффективность, расходится с остальным кодом
S1	<code>hls.js</code> закрепляется «доверием при первом скачивании»	СРЕД	Свежее развёртывание принимает любой файл, отданный CDN в тот момент
S2	Пин <code>dev-зависимости</code> блокирует единственное исправление CVE	НИЗК	Следующий релиз по тегу упадёт на собственной проверке <code>pip-audit</code>
S3	<code>audit_log</code> растёт без ограничений и защищён от удаления	СРЕД	Строка на каждый просмотр по ссылке; триггер запрещает DELETE — штатного способа подрезать таблицу нет
S4	Непоследовательное экранирование в шаблоне письма	НИЗК	Латентная HTML-инъекция в письмо для следующего, кто добавит вызов
S5	<code>api</code> доверяет X-Real-IP от всех соседей по сети edge	НИЗК	Эшелонированная защита: скомпрометированный MediaMTX сможет подделать IP
S6	Нет CSRF-токена на <code>/logout</code>	ИНФО	Закрывается <code>SameSite=Lax</code> ; добавление зависимости ничего не стоит
S7	Документация лимитов расходится с реализацией	ИНФО	Окно фиксированное, а не скользящее: на стыке окон проходит до 2× лимита

ID	Находка	Важность	Что это значит на практике
S8	Сброс 2FA администратором не требует подтверждения	ИНФО	Один скомпрометированный админ-сеанс снимает второй фактор с любого

3. Баги и необработанные граничные случаи

В1. TTL множества зрителей ссылки не продлевается

СРЕД

backend/app/internal/authz.py:93 (grant) ⇄ :304–315 (_allow)

grant() заводит два ключа с одинаковым сроком жизни — viewer:<vid> (права зрителя) и link_viewers:<link_id> (множество, на котором держатся лимит и метрика). Но продлевает _allow() только первый:

```
# grant() – оба ключа получают ttl_seconds (по умолчанию 60 минут)
pipe.expire(viewer_key, ttl_seconds)
pipe.expire(link_viewers_key, ttl_seconds)      # ← ставится один раз

# _allow() – вызывается на КАЖДЫЙ медиа-запрос, продлевает только права
await get_redis().expire(f"_{VIEWER_PREFIX}{viewer_id}", ttl)
#                               ^^^^^^^^^^^^^^^^^^^^^ link_viewers здесь нет
```

Зритель, который час не перезагружал страницу, продолжает смотреть (права продлеваются), но выпадает из множества. Последствия:

- **Лимит одновременных зрителей перестаёт работать.** count_viewers() вернёт 0, и ссылка с max_concurrent=1 впустит второго, третьего и далее без ограничения.
- Метрика rtspgw_active_viewers занижает число зрителей — ровно та ошибка, ради исправления которой её и переносили из view_sessions в Redis.
- viewer_counted() вернёт False, и перезагрузка страницы снова займёт слот в лимите.

Отзыв ссылки при этом не страдает: он удаляет ключ кэша решения, и следующий же запрос идёт в БД. Но это стоит проверить тестом — сейчас корректность отзыва держится на invalidate_link, а не на drop_link_viewers, который в этом состоянии вхолостую перебирает пустое множество.

Как чинить. В _allow() идентификатор ссылки уже есть параметром — продлевать оба ключа одним конвейером:

```
pipe = get_redis().pipeline()
pipe.expire(f"_{VIEWER_PREFIX}{viewer_id}", ttl)
if link_id != OPERATOR_GRANT:
    pipe.expire(f"_{LINK_VIEWERS_PREFIX}{link_id}", ttl)
await pipe.execute()
```

В2. Блокировка PTZ не снимается при неизвестном направлении

НИЗК

backend/app/ptz/service.py:253–267

press() сначала захватывает камеру, и только потом разбирает направление. На неизвестном значении управление остаётся захваченным на ptz_hold_seconds:

```
acquired, retry_after = await lock.acquire(camera.id, holder, ...) # :253
if not acquired:
    return Result(ok=False, reason='busy', ...)

try:
    velocity = Vector.from_direction(direction) # :265
except ValueError:
    return Result(ok=False, reason='error',
                  message='Неизвестное направление.') # :267
    # ← lock.release() НЕ вызывается: камера занята ещё 15 секунд

except (PtzError, UnsafeCameraUrl, DecryptionError) as exc: # :275
    await lock.release(camera.id, holder)
```

Это противоречит инварианту, который в этом же файле сформулирован явно десятью строками ниже: *«Неисправная камера не должна держать управление за собой: иначе следующие пятнадцать секунд*

пульт заблокирован ни для кого». Практическое влияние небольшое — зритель, который может слать мусор, может слать и корректные команды, — но это дыра в заявленном свойстве.

Как чинить. `Vector.from_direction()` — чистая функция и не требует блокировки. Перенести её вызов *выше* `lock.acquire()`: тогда мусорный запрос отбивается вообще без обращения к Redis.

В3. Счётчик просмотров ссылки инкрементируется через гонку

НИЗК

backend/app/web/public_views.py:123

`link.view_count += 1` — это read-modify-write на стороне Python. Два одновременных открытия ссылки читают одно и то же значение и записывают одно и то же `n+1`: один инкремент теряется. Для популярной ссылки, которую открывают из рассылки, расхождение будет заметным.

Как чинить. Инкремент на стороне БД, атомарно:

```
await db.execute(
    update(ShareLink)
    .where(ShareLink.id == link.id)
    .values(view_count=ShareLink.view_count + 1,
            last_viewed_at=dt.datetime.now(dt.UTC))
)
```

В4. Ответ камеры читается целиком, ограничение размера мнимое

НИЗК

backend/app/ptz/onvif.py:61,264,281 · backend/app/ptz/vendors.py:24,60,141,195

Оба драйверных модуля объявляют предел размера тела и комментируют его как защиту — *«Дальше этого тела ответа не читаем — защита от бесконечного потока»*. Фактически защиты нет:

```
_MAX_BODY = 256_000
...
text = response.text[:_MAX_BODY]    # ← срез ПОСЛЕ полного чтения
```

httpx без `stream()` буферизует ответ целиком ещё до обращения к `.text`, и своего лимита на размер тела у него нет. Камера (или что угодно, что оказалось на её адресе и порту) может вернуть многогигабайтный ответ, и весь он окажется в памяти процесса `api` — того самого, который обслуживает панель.

Сценарий не гипотетический: адрес PTZ — это `camera.host` с произвольным портом, и оператор легко направит опознание на HTTP-сервер, отдающий большой файл.

Как чинить. В `transport.request()` перейти на потоковое чтение с жёстким пределом:

```
async with get_client().stream(method, url, ...) as response:
    chunks, total = [], 0
    async for chunk in response.aiter_bytes():
        total += len(chunk)
        if total > _MAX_BODY:
            raise PtzError('камера вернула слишком большой ответ')
        chunks.append(chunk)
```

В5. Гонка при одновременном принятии приглашения

НИЗК

backend/app/invites.py:184 (accept) · backend/app/web/invite_views.py

`accept()` проверяет отсутствие пользователя с таким адресом обычным `SELECT`, а затем вставляет строку. Между проверкой и вставкой нет ни блокировки строки приглашения, ни обработки конфликта. Два одновременных `POST` по одной ссылке (двойной клик, повтор из офлайн-очереди браузера) проходят проверку оба; второй падает на уникальном индексе `users.email`.

`IntegrityError` — не `InviteError`, поэтому обработчик его не ловит: сотрудник вместо понятного текста получает страницу «Внутренняя ошибка сервиса» на первом же шаге знакомства с сервисом. Учётная запись при этом создана, и он этого не знает.

Как чинить. Взять приглашение `SELECT ... FOR UPDATE` в `resolve()` либо поймать `IntegrityError` в `invite_accept` и показать то же сообщение, что и для уже существующей учётки.

В6. Заблокированная учётная запись отличима по времени ответа

[ИНФО](#)

`backend/app/web/auth_views.py:120–145`

Форма входа аккуратно закрывает утечку существования учётки: неизвестному адресу считается `argon2` по заглушке (`_DUMMY_HASH`), текст ошибки один и тот же. Но ветка «учётка заблокирована» возвращается **до** любой проверки пароля — то есть за единицы миллисекунд вместо ~60 мс, с другим HTTP-кодом (429) и другим текстом. Это восстанавливает ровно тот сигнал, который остальной код прячет.

Смежное наблюдение — не дефект, а компромисс, о котором стоит знать: правило `MAX_FAILED_ATTEMPTS = 10` → блокировка на 15 минут делает возможным адресный отказ в обслуживании. Тот, кто знает рабочую почту коллеги, может держать его заблокированным бесконечно, тратя 10 запросов раз в четверть часа.

Как чинить. Считать заглушку и в этой ветке — одна строка. Для блокировки рассмотреть привязку к паре «учётка + IP» либо экспоненциальную задержку ответа вместо жёсткого запрета входа.

4. Производительность

P1. Эндпоинт /metrics сканирует весь keyspace Redis

СРЕД

backend/app/internal/authz.py:116–128 · backend/app/main.py (metrics)

```
async def count_all_viewers() -> int:
    total = 0
    async for key in redis.scan_iter(match=f'_{LINK_VIEWERS_PREFIX}*', count=200):
        total += int(await redis.scard(key))
    return total
```

SCAN с MATCH фильтрует **после** обхода: стоимость определяется размером всего keyspace, а не числом подходящих ключей. А в этом keyspace лежат не только зрители: sess:*, user_sess:*, r1:* (по ключу на каждый IP и каждую учётку в каждом окне), viewer:*, authz:link:*, totp_used:*, ptz:*. На живой инсталляции это легко десятки тысяч ключей.

Prometheus в ops/prometheus/prometheus.yml опрашивает с интервалом 15 с — значит полный обход keyspace идёт четыре раза в минуту, круглосуточно, плюс отдельный round-trip SCARD на каждый найденный ключ. Комментарий в коде оправдывает подход тем, что «при интервале сбора 15 с это дешевле, чем держать счётчики в памяти» — для SQL-запросов рядом это верно, а для SCAN — нет: он единственный в этом эндпоинте растёт вместе со всей нагрузкой сервиса, а не с числом камер.

Как чинить. Держать реестр действующих ссылок отдельным множеством (SADD в grant(), SREM в drop_link_viewers()) и обходить только его. Дешевле и проще — кэшировать значение метрики на длительность интервала сбора.

P2. Главный цикл worker'a блокируется внешними процессами

СРЕД

backend/app/worker.py:165–185

Циклы выполняются строго последовательно, и следующий тик начинается только после самого медленного из них:

```
for name, coro in (('reconcile', _reconcile_cycle()),
                  ('probe', _probe_cycle())):
    await coro
...
await asyncio.wait_for(stop.wait(), timeout=settings.reconcile_interval_seconds)
```

_probe_cycle(limit=3) для каждой новой камеры запускает ffprobe (таймаут 20 с) и следом capture_snapshot с ffmpeg (таймаут 25 с) — тоже последовательно. Худший случай — **около 135 секунд** на один тик при заявленном reconcile_interval_seconds = 15.

Это ровно тот момент, когда реконсильация нужна больше всего: массовое добавление камер, восстановление после перезапуска MediaMTX. Пока worker ждёт три неотвечающие камеры, пути в MediaMTX не создаются и статусы в панели не обновляются — оператор видит «не проверена» на камерах, которые давно в эфире. Раз в 20 тиков к этому добавляются _snapshot_cycle и _cleanup_cycle, причём снапшоты снимаются со **всех** онлайн-камер подряд, тоже по одной.

Как чинить. Развести циклы по независимым задачам (asyncio.TaskGroup) со своими интервалами: реконсильация обязана идти по расписанию независимо от того, сколько сейчас пробуете камер. Внутри пробы и снапшотов — ограниченный параллелизм через asyncio.Semaphore (процессы всё равно ждут сеть, а не CPU).

P3. Списочные страницы без пагинации

НИЗК

panel_views.py (dashboard, _render_detail) · admin_views.py (_users_page)

Журнал аудита сделан образцово — курсорная пагинация по составному индексу (created_at, id), с честным разбором, почему не OFFSET. Соседние страницы этого не получили и грузят выборку целиком:

- `dashboard` — все камеры пользователя плюс агрегат по ссылкам; страница рендерит карточку с превью на каждую.
- `_render_detail` — все ссылки камеры, включая давно отозванные (они не удаляются и не архивируются).
- `_users_page` — все пользователи и все неиспользованные приглашения.

На десятках объектов это незаметно, на сотнях — заметно, и первым просядет именно список ссылок: он растёт быстрее всех и никогда не подрезается.

P4. Разбор CIDR на каждом медиа-запросе

НИЗК

`backend/app/internal/authz.py:170-182`

`ip_allowed()` вызывает `ipaddress.ip_network()` для каждой строки списка на каждом вызове. Вызывается он из `authz` — то есть на каждый сегмент LL-HLS, несколько раз в секунду на каждого зрителя. Разбор строки в объект сети в Python не бесплатен, и это единственная вычислительная работа на «горячем» пути, который в остальном сводится к двум обращениям к Redis.

Как чинить. Кэш решения (`LinkAccess`) и так десериализуется на каждом запросе — разбирать подсети там же, один раз, и хранить в `LinkAccess` уже готовые объекты сетей.

P5. Новый HTTP-клиент на каждый зонд диагностики

НИЗК

`backend/app/media/diagnose.py (_probe_endpoint)`

`_probe_endpoint` создаёт `httpx.AsyncClient` внутри `async with` на каждый вызов — то есть новый пул соединений и новый TLS-контекст. Для MediaMTX и PTZ в проекте заведены аккуратные клиенты-синглтоны с явным закрытием в `lifespan`; здесь эта линия прервана.

Диагностика запускается человеком и редко, так что влияние минимальное — находка про единообразие, а не про нагрузку.

5. Безопасность

Разделено намеренно: здесь — непреднамеренные слабости, в разделе 6 — компромиссы, о которых автор знает и которые описал в коде и `docs/security.md`. Обхода аутентификации, доступа к чужим камерам, SQL-инъекций, XSS и утечки секретов в логи не обнаружено.

S1. `hls.js` закрепляется «доверием при первом скачивании»

СРЕД

`ops/fetch-vendor.sh:28-42` · `backend/app/web/static/vendor/`

Скрипт качает единственную стороннюю JS-библиотеку проекта и сверяет её контрольную сумму — но эталон он **сам же и создаёт** при первом запуске:

```
if [ -f "$SUMFILE" ]; then
    expected="$(cat "$SUMFILE")"
    [ "$actual" != "$expected" ] && exit 1
else
    echo "$actual" > "$SUMFILE"          # ← эталон = то, что сейчас отдал CDN
fi
```

Каталог `vendor/` в репозитории содержит только `README.md` — файла с суммой там нет (проверено: `git ls-files` возвращает одну строку). Значит **каждое новое развёртывание** — установка на чистый сервер, пересборка в CI, клон коллеги — принимает как эталон то, что `jsdelivr` отдал в этот момент. Сверка появляется только начиная со второго запуска на той же машине, то есть тогда, когда она уже почти не нужна.

Библиотека попадает в образ и исполняется в браузере зрителя на странице, где лежит cookie доступа к медиа. CSP (`script-src 'self'`) от подменённого локального файла не защищает — он и есть `'self'`.

Как чинить. Записать эталонную сумму для закреплённой версии `HLS_VERSION` в репозиторий и сделать ветку «эталона нет» ошибкой, а не тихим созданием файла. Проверка занимает одну строку и превращает TOFU в настоящий пин.

S2. Пин dev-зависимости блокирует единственное исправление CVE

НИЗК

`backend/pyproject.toml (optional-dependencies.dev)` · `.github/workflows/ci.yml`

`pip-audit` находит PYSEC-2026-1845 в `pytest 8.4.2` (локальная эскалация через предсказуемые каталоги в `/tmp`). Исправление — 9.0.3. Проект пинует:

```
"pytest>=8.3,<9.0",          # ← верхняя граница отсекает 9.0.3
```

Сама уязвимость влияния на продукт не имеет: `pytest` в образ не ставится (`Dockerfile` выполняет `pip install . без dev-extra`), а эксплуатация требует локального пользователя на машине сборки.

Важнее последствие для процесса. CI устроен так, что на обычном пуше находка справочная, а на теге — блокирующая:

```
- name: Уязвимости в зависимостях
  run: pip-audit --strict --desc
  continue-on-error: ${ !startsWith(github.ref, 'refs/tags/') }
```

То есть **следующий релиз по тегу упадёт на собственной проверке**, и починить это правкой окружения нельзя — мешает пин в `pyproject.toml`. Замысел проверки правильный, но он сработает как ложная тревога ровно в тот момент, когда к нему должно быть больше всего доверия.

Как чинить. Поднять до `"pytest>=9.0.3,<10.0"`. Заодно стоит закрепить в CI и сам факт установки `extra fakeredis[lua]` — без него 10 тестов блокировки PTZ не выполняются (см. раздел 1).

S3. `audit_log` растёт без ограничений и защищён от удаления

СРЕД

`backend/alembic/versions/20260902_0001_initial_schema.py` · `backend/app/worker.py:126-150`

Миграция ставит на таблицу триггер, запрещающий любые UPDATE и DELETE:

```
CREATE TRIGGER audit_log_no_update_delete
BEFORE UPDATE OR DELETE ON audit_log
FOR EACH ROW EXECUTE FUNCTION audit_log_is_append_only();
-- функция безусловно делает RAISE EXCEPTION
```

Решение правильное: журнал должен быть неизменяемым. Проблема в том, что `_cleanup_cycle()` подрезает `view_sessions` (90 дней) и `invitations` (30 дней), а для `audit_log` срока хранения нет вообще. При этом строка пишется на **каждое** открытие публичной ссылки (`link.viewed`) и на каждый отказ (`link.denied`) — то есть на ту же частоту событий, из-за которой `view_sessions` названа в комментарии «самой быстрорастущей в схеме».

Комментарий в `worker`'е прямо перекладывает историю на этот журнал: «Кто и когда открывал ссылку, остаётся в `audit_log` — там это и положено искать». Получается таблица, которая растёт быстрее подрезаемой, не подрезается никогда и **не может** быть подрезана штатно: DELETE поднимает исключение. Оператор, упёршийся в диск, вынужден снимать триггер — то есть отключать контроль целостности ради уборки.

Как чинить. Секционировать таблицу по месяцам: DETACH PARTITION + DROP TABLE не задевает построчный триггер, и архивация становится штатной операцией. Более простой вариант — функция SECURITY DEFINER, выставляющая флаг сессии, который триггер уважает, плюс срок хранения рядом с остальными константами в `worker.py`. И отдельно стоит подумать, обязана ли каждая перезагрузка страницы зрителем порождать запись в юридически значимом журнале.

S4. Непоследовательное экранирование в шаблоне письма

НИЗК

backend/app/mail.py:197-272 (wrap_html)

Функция принимает шесть текстовых фрагментов и экранирует ровно два из них:

<code>{html.escape(greeting)}</code>	# :235	экранируется
<code>{html.escape(button_label)}</code>	#	экранируется
<code>{lead}</code>	# :238	вставляется как есть
<code>{fine_print}</code>	# :254	вставляется как есть
<code>{footer}</code>	# :270	вставляется как есть

Сейчас это безопасно: единственный вызов, куда попадают пользовательские данные — `invites._render_email`, — экранирует имя приглашающего вручную (`html.escape(who)`), а `recovery.render_email` передаёт константы. Но контракт функции нигде не записан, и он противоречив: соседние параметры одного и того же вызова ведут себя по-разному.

Следующий, кто добавит в письмо название камеры или комментарий администратора через `lead`, получит HTML-инъекцию в письмо — а письмо от сервиса читают вне контекста CSP, и подделать в нём кнопку «Задать пароль» стоит одной строки.

Как чинить. Экранировать все шесть внутри `wrap_html`, а для фрагментов, которым разметка нужна намеренно, ввести явный тип (`markupsafe.Markup`) — тогда контракт проверяет туру, а не память.

S5. api доверяет X-Real-IP от всех соседей по сети edge

НИЗК

backend/app/middleware.py (client_ip) · docker-compose.yml

`client_ip()` берёт X-Real-IP без всяких условий, и обоснование в комментарии звучит так: «Заголовку доверяем только потому, что порт 8000 не опубликован наружу и достучаться до него, минуя Caddy, нельзя». Для хоста это верно. Внутри docker — нет:

```
api:      networks: [edge, core, data, inet]
mediamtx: networks: [edge, core]           # ← общая сеть с api
command:  ... --forwarded-allow-ips=*
```

`mediamtx` делит с `api` сразу две сети, а именно он единственный контейнер, принимающий трафик с публичного порта (8189/udp и 8189/tcp) — то есть первый кандидат на компрометацию. Из него `api:8000`

доступен напрямую, и подделанный X-Real-IP обходит ограничение ссылки по подсети, обнуляет лимиты частоты по IP и записывает чужой адрес в журнал аудита.

Это именно эшелонированная защита: для реализации нужен уже взломанный медиа-сервер. Но Caddyfile тратит два абзаца на объяснение, почему `trusted_proxies` нельзя задавать диапазоном, — а на последнем участке цепочки та же проверка отсутствует.

Как чинить. Заменить `--forwarded-allow-ips=*` на адрес Caddy и сверять источник в `client_ip()`; либо вывести `mediamtx` из `edge` в отдельную сеть, общую только с Caddy.

S6. Нет CSRF-токена на /logout

ИНФО

backend/app/web/auth_views.py:398

Все изменяющие состояние обработчики панели объявляют зависимость `CsrfProtected` — кроме `logout`. Практической эксплуатации нет: cookie сессии помечена `SameSite=Lax`, и межсайтовый POST её просто не донесёт. Но защита от CSRF здесь держится на одном рубеже вместо двух, а исключение из общего правила ничем не объяснено — в проекте, где объяснено буквально всё, это читается как недосмотр. Добавить `_: CsrfProtected` — одна строка.

Формы `/login`, `/forgot`, `/reset/{token}` и `/invite/{token}` токена не имеют обоснованно — сессии на них ещё нет, и для трёх последних роль неподделяемого секрета играет сам токен в адресе. Это отмечено в комментарии к `invite_views`.

S7. Документация лимитов частоты расходится с реализацией

ИНФО

backend/app/auth/ratelimit.py

Модуль называется «Ограничение частоты запросов (скользящее окно на счётчиках Redis)», и `hit()` повторяет это в докстринге. Реализация — фиксированное окно: `SET NX EX` заводит счётчик со сроком, `INCR` его увеличивает, по истечении срока всё обнуляется разом.

Разница практическая: на стыке двух окон проходит до двойного лимита (20 попыток входа в последнюю секунду первого окна и ещё 20 в первую секунду второго). Для выбранных значений это приемлемо и вряд ли стоит переделки — но описание стоит привести в соответствие, иначе следующий читатель будет считать защиту строже, чем она есть.

Отдельно стоит отметить: сама реализация `hit()` сделана правильно и с разбором именно тех граблей, на которые тут обычно наступают — атомарность создания окна и восстановление TTL у ключа, оставшегося без срока.

S8. Сброс второго фактора администратором не требует подтверждения

ИНФО

backend/app/web/admin_views.py (user_reset_totp)

Обработчик сам называет операцию опасной и пишет её в аудит — это правильно. Но выполняется она одним POST: без ввода пароля администратора и без ограничения частоты. Один угнанный сеанс администратора снимает второй фактор со всех учётных записей подряд, оставляя вход защищённым только паролем.

Смежное: в отличие от `user_toggle` и `user_role`, здесь нет проверки `target.id == admin.id` — администратор может сбросить 2FA сам себе в обход политики `totp_policy=admins`, которую `totp_disable` в профиле специально не даёт обойти. Это дыра именно в политике, а не в правах.

Как чинить. Запрашивать пароль администратора (шаблон уже есть — так устроено отключение 2FA в профиле) и не давать сбрасывать себе, когда политика требует второй фактор для этой роли.

6. Осознанно принятые риски (не дефекты)

Эти решения приняты сознательно и объяснены в коде и документации. Перечислены, чтобы отделить их от находок выше и чтобы при следующем ревью они не открывались заново; для части из них есть смысл в дополнительном рубеже.

Компромисс	Обоснование автора	Комментарий ревьюера
verify=False для HTTP к камере ptz/transport.py	Сертификат IP-камеры почти всегда самоподписанный; выбор между «без проверки цепочки» и «PTZ по HTTPS не работает никогда». Пароль не идёт открытым — везде Digest или ONVIF-дайджест	Согласен. Стоит добавить необязательный отпечаток сертификата на камеру: тогда те, кто может, получат настоящую проверку, а остальные — прежнее поведение. Сейчас атакующий в сегменте камер видит трафик управления и может собрать Digest-челленджи для офлайн-подбора
DNS rebinding между проверкой и подключением media/ssrf.py	Закрывается правилами egress на хосте, см. docs/security.md	Риск назван честно. Усиление без egress-правил: сохранять проверенный IP и передавать в MediaMTX/ffmpeg именно его, оставляя имя только для TLS. Сейчас и MediaMTX, и ffmpeg резолвят имя заново, так что проверенные адреса носят справочный характер
Сторож PTZ не переживает перезапуск api ptz/service.py	Камеры Dahua и Axis доедут до упора, если убить процесс в момент удержания стрелки. Случай редкий и самоустраняющийся; надёжное решение потребовало бы состояния в Redis	Согласен, цена выше пользы. Отмечено в docs/runbook.md — этого достаточно
MediaMTX без пароля , доступ по подсетям mediamtx/mediamtx.yml	Секрет в примонтированном read-only файле было бы нечем ротировать; разделение подсетей даёт тот же эффект	Приемлемо при текущей сетевой схеме. Связано с S5: обе находки упираются в то, что edge общая для Caddy, api и mediamtx
Cookie просмотра SameSite=None	Иначе не работает встраивание через /embed/; CSRF публичного пульта закрыт preflight'ом и заголовком X-Requested-With	Рассуждение верное, обходного пути нет: CORS не разрешён нигде, чужая страница preflight не пройдёт. Cookie сессии панели осталась lax — разделение корректное
Токен ссылки возвращается в форму при повторном показе panel_views._camera_form_val ues	Цена «не возвращать» выше пользы: после каждой проверки оператор вставлял адрес заново. Ответы идут с Cache-Control: no-store	Согласен. Пароль камеры и так виден тому же человеку, который его секунду назад ввёл

7. Что сделано заметно лучше обычного

Перечислено не из вежливости: эти решения стоит сохранить при рефакторинге, потому что каждое из них закрывает ошибку, которую в проектах этого класса допускают регулярно.

Область	Что именно
Комментарии	Объясняют причину , а не пересказывают код, и почти всегда называют конкретную поломку, из-за которой решение такое. Комментарий про route в Caddyfile или про порядок middleware в create_app экономит следующему человеку полдня отладки. Это ощутимо выше отраслевой нормы
Модель угроз	Токены хранятся хешами, отзыв мгновенный и не требует списков отзыва (в отличие от JWT), пути MediaMTX неугадываемы, forward_auth не полагается на префикс URL, кэш решения инвалидируется явно. Формулировки отказов единообразны и не выдают, существует ли объект

Область	Что именно
Криптография	argon2id с параметрами выше умолчаний, SecretBox для учётных данных камер и TOTP-секретов, версионированный формат blob'a и рабочая команда ротации ключа (включая отдельные креды PTZ — их легко забыть), защита TOTP от повторного использования кода через SET NX, compare_digest везде, где сравниваются секреты
Отказоустойчивость	MediaMTX не хранит состояния, реконсиллятор восстанавливает конфигурацию из PostgreSQL — перезапуск и обновление медиа-сервера не требуют действий. Vackoff при пересоздании пути по степеням, а не в каждом цикле
Тесты	460 тестов без Postgres и Redis: схема сверяется с миграциями офлайн через рендер SQL, Redis подменяется in-memory. Отдельно проверяются порядок директив в Caddyfile, стили, шаблоны и разбор ввода установщика — то есть ровно те места, где обычно ломается молча
Диагностика	media/diagnose.py — сильнейшая часть проекта с точки зрения эксплуатации. Шаги повторяют путь пакета, ключевой шаг заставляет MediaMTX реально подключиться, а не полагается на ffprobe, и пароль вырезается из вывода регуляркой по произвольной строке, а не через urlsplit
Сопровождается	Ошибки пользователю написаны на языке действия («включите HTTPS в веб-интерфейсе камеры»), а не на языке протокола. Сверка схемы БД с миграциями при старте и в /readyz. Русские числительные в интерфейсе. Журнал append-only на уровне БД, а не на уровне договорённости

8. Чего проекту не хватает

Разделено по горизонту. Первая группа — то, что я бы сделал сразу после исправления находок; вторая — заметные функциональные пробелы; третья — направления развития.

8.1. Ближний горизонт: доработать начатое

Предложение	Почему
Срок хранения и архивация журнала аудита	Прямое следствие S3. Пока это единственная таблица без ретенции, и она же самая быстрорастущая
Страница «кто смотрит прямо сейчас»	Данные уже есть в Redis — множества <code>link_viewers</code> , — но используются только метрикой. Для сервиса, весь смысл которого в раздаче доступа наружу, «кто открыл мою камеру в эту минуту» — первый вопрос владельца. Заодно даёт естественный способ проверять B1
Кнопка «оборвать этого зрителя»	Механика уже написана: <code>drop_link_viewers</code> и <code>_kick_active_sessions</code> отзывают доступ по ссылке целиком. Точечный отзыв — небольшая надстройка над тем же кодом
Уведомление на почту о падении камеры	SMTP настроен, шаблон письма есть, <code>failure_streak</code> считается, алерты Prometheus описаны. Не хватает только связки — а без неё про упавшую камеру узнают, когда она понадобится
Пагинация списков	P3. Курсорная реализация уже написана для журнала — её можно переиспользовать
Ротация сессий при включении 2FA	При смене уровня привилегий сессия ротируется (<code>rotate()</code> после ввода кода), а при включении второго фактора в профиле — нет. Логично завершить и остальные сеансы пользователя, как это уже делается при смене пароля

8.2. Функциональные пробелы

Функция	Комментарий
Запись и архив	Самый крупный пробел для продукта про камеры: <code>record: false</code> зашито и в <code>pathDefaults</code> , и в <code>build_path_conf</code> . Живой просмотр отвечает на вопрос «что происходит», но не на «что произошло», а второй задают чаще. MediaMTX умеет запись и <code>playback</code> из коробки — нужна модель хранения, ретенция, квоты и права на просмотр архива. Это следующий большой шаг, и его стоит спланировать до того, как схема БД обростёт зависимостями
Пресеты PTZ	Драйверы уже говорят на всех четырёх протоколах, а пресеты — самая востребованная функция после стрелок: «показать ворота», «показать кассу». У ONVIF это <code>GotoPreset</code> , у остальных — один дополнительный запрос. Дешевле, чем выглядит
Многоузловость доведена до половины	<code>Camera.node_id</code> есть в схеме, <code>reconcile</code> и <code>refresh_statuses</code> по нему фильтруют — но значение везде жёстко <code>"default"</code> , распределения камер по узлам нет, и как узел объявляет о себе, не описано. Задел полезный, но в текущем виде он выглядит работающим, не будучи им; стоит либо довести, либо отметить в комментарии как незавершённый
Массовые операции	Импорт камер из CSV, отзыв всех ссылок камеры одной кнопкой, включение и выключение группы. Первая же установка на объект с полусотней камер упрётся в это
Экспорт журнала	Журнал читают в основном при разборе инцидента, и там он нужен файлом, а не страницей. CSV с тем же фильтром, что на экране
Обновление превью по кнопке	У камер <code>on-demand</code> снимок делается один раз при добавлении — намеренно, чтобы не будить поток. Кнопка «обновить кадр» закрывает случай «превью годовой давности, а камеру давно развернули»

8.3. Направления развития

- **Passkeys / WebAuthn как второй фактор.** TOTP реализован полно (включая коды восстановления и защиту от повтора), и следующий естественный шаг — аппаратный фактор. Для сервиса, раздающего доступ к видеонаблюдению, это уместно, а инфраструктура ротации сессий и политики `totp_policy` уже есть.
- **Ограниченный API для интеграций.** OpenAPI сейчас отключён намеренно («сервис не для внешних интеграций»). Но выдача ссылки по событию из СКУД или встраивание списка камер в существующий NOC — типовой корпоративный запрос, и решать его подделкой формы никто не должен. Токены с ограниченной областью действия и отдельным журналом.
- **Квоты на ссылки.** Сейчас оператор может выпустить неограниченное число бессрочных ссылок. Лимит на камеру или на пользователя плюс отчёт «ссылки, которые никто не открывал 90 дней» — дешёвая гигиена для того, что и является главным активом сервиса.
- **Проверка восстановления из копии в CI.** `ops/backup.sh` честно проверяет, что дамп распаковывается и не подозрительно мал, — это уже больше, чем обычно делают. Логичное продолжение — периодический прогон `restore.sh` в одноразовое окружение: непроверенная резервная копия остаётся гипотезой.
- **Метрики бизнес-уровня.** Сейчас в `/metrics` три показателя. Полезны также распределение задержки `forward_auth`, доля переключений на HLS-фолбэк (прямой индикатор проблем с UDP у зрителей) и число отказов по причинам — последнее уже пишется в лог функцией `_deny`.

9. Приложение: как воспроизвести проверки

```
cd backend
python -m venv .venv
.venv/bin/pip install -e ".[dev]"      # extra [lua] обязателен для тестов PTZ

.venv/bin/python -m ruff check app alembic tests
.venv/bin/python -m mypy app
.venv/bin/python -m pytest -q
.venv/bin/python -m pip_audit --desc

cd .. && bash tests/test_install_sh.sh
```

Все находки получены чтением исходного кода и запуском перечисленных проверок. Развёртывание не выполнялось: `docker` на машине ревьюера отсутствует, поэтому поведение `Caddy`, `MediaMTX` и цепочки `forward_auth` оценивалось по конфигурации и тестам `test_caddy_order.py` и `test_deployment_config.py`, а не наблюдением на живом стенде. Утверждения о поведении под нагрузкой (P1, P2) выведены из кода и заявленных интервалов, а не измерены.